

DEVOIR PRATIQUE N°3

NLP avec PyTorch

Classification de texte : Fine-tuning de BERT

Entraînement PyTorch + Démo Gradio - Travail en binôme (2 personnes)

Niveau	Master / Ingénierie IA - NLP & Deep Learning
Objectif	Fine-tuner un modèle BERT pour la classification de texte et déployer une démo interactive
Modalité	Travail en groupe de 2 personnes (binôme obligatoire)
Outils	Python 3.9+, PyTorch ≥2.0, transformers (Hugging Face), Gradio, scikit-learn GPU recommandé (Google Colab accepté)
Date limite	Mardi 16 Juin 2026 - 18h00 Lien GitHub à envoyer à : abdouaziz@gmail.com

1. Contexte et Dataset

Après avoir exploré les architectures CNN pour la vision (Devoir n°2), ce troisième devoir vous amène dans le domaine du NLP (Natural Language Processing). Vous allez fine-tuner un modèle **BERT** (Bidirectional Encoder Representations from Transformers) pré-entraîné pour une tâche de **classification de texte**, en implémentant la boucle d'entraînement en PyTorch, puis vous déploierez une interface de démonstration interactive avec **Gradio**.

Dataset - Choix du sujet (un dataset par binôme)

URL : <https://docs.google.com/spreadsheets/d/15Tgadm2s1uZze1tS7SBCK5vMYXQBfpCTx5DTzvsDnE/edit?usp=sharing>

Chaque binôme choisit **un dataset** dans la feuille de calcul partagée ci-dessus et inscrit le nom des deux membres du groupe en face du dataset choisi.

Règle : premier arrivé, premier servi - deux binômes ne peuvent pas choisir le même dataset.

Avant d'écrire la moindre ligne de code, inspectez votre dataset :

- Nombre total d'exemples et nombre de classes
- Distribution des classes - si déséquilibre > 2:1, justifiez votre stratégie
- Longueur des textes (min, max, moyenne en nombre de tokens) - pour choisir max_length
- Afficher au moins 5 exemples de textes avec leurs labels

2. Objectifs Pédagogiques

Ce devoir évalue les compétences suivantes :

Compétences techniques

- Charger et tokenizer un dataset texte avec la librairie transformers (Hugging Face)
- Implémenter une classe Dataset PyTorch personnalisée pour le texte
- Fine-tuner un modèle BERT pré-entraîné avec une boucle d'entraînement PyTorch pure
- Construire et déployer une interface de démonstration avec Gradio
- Versionner et documenter un projet sur GitHub (travail collaboratif en binôme)

Compétences analytiques

- Analyser les courbes d'apprentissage (overfitting, underfitting)
- Interpréter les métriques : accuracy, F1-score, matrice de confusion
- Comprendre l'apport du pré-entraînement (transfer learning) en NLP
- Rédiger une documentation technique claire (README.md)

3. Travail à Réaliser

3.1 Étape 1 : Créer un Repo GitHub (obligatoire)

Créez un dépôt GitHub **public** pour votre binôme. Ce dépôt servira de livrable unique : c'est son lien que vous enverrez par email.

- Nom explicite, ex. : bert-classification-nomdataset
- Les **deux membres** du binôme doivent être collaborateurs et avoir des commits visibles
- Historique de commits propre et progressif (pas un seul commit final !)
- Un fichier `requirements.txt` listant les dépendances
- Un fichier `README.md` (voir section Bonus)

3.2 Étape 2 : Entraînement du modèle avec PyTorch (obligatoire)

Implémentez le fine-tuning d'un modèle BERT pré-entraîné (ex. bert-base-uncased, bert-base-multilingual-cased ou camembert-base selon la langue de votre dataset) pour la classification de votre dataset.

Votre implémentation doit contenir :

1. Le chargement du tokenizer et du modèle pré-entraîné via la librairie transformers (Hugging Face)
2. Une classe Dataset PyTorch personnalisée pour vos textes (tokenization, padding, attention mask)
3. Un split train / validation 80/20 stratifié
4. Des fonctions d'entraînement et d'évaluation retournant loss, accuracy et F1-score
5. La sauvegarde du meilleur modèle (best val_loss)

! Contrainte importante

Vous ne devez **pas** utiliser le Trainer de Hugging Face.

La boucle d'entraînement doit être écrite manuellement en PyTorch, comme dans le Devoir n°2.

Pensez à bien basculer le modèle en mode évaluation et à désactiver le calcul des gradients pendant la validation.

Métriques à collecter à chaque epoch

- train_loss • val_loss
- train_accuracy • val_accuracy
- val_f1_score • learning_rate (si scheduler)

3.3 Étape 3 : Interface de démo avec Gradio (obligatoire)

Créez un fichier `demo.py` qui charge votre meilleur modèle sauvegardé et expose une interface web interactive avec **Gradio** (documentation : <https://www.gradio.app/>).

L'interface doit :

- Accepter un texte saisi par l'utilisateur
- Afficher la classe prédite **et** les probabilités de chaque classe
- Avoir un titre, une description et au moins 2 exemples pré-remplis
- Se lancer simplement avec : `python demo.py`

Rappel : Gradio

Gradio permet de créer une interface web de démonstration en quelques lignes de Python.

Consultez la documentation officielle et les guides de démarrage : <https://www.gradio.app/>

Le code de l'interface doit être écrit par votre binôme - aucun squelette n'est fourni.

3.4 Bonus : Rapport détaillé dans le README.md

Des points bonus seront attribués si votre `README.md` fournit un véritable rapport sur la réalisation des étapes, comprenant :

- Présentation du dataset choisi (source, classes, statistiques, exemples)
- Description du modèle et des choix techniques (tokenizer, `max_length`, tête de classification)
- Détail des étapes de réalisation et difficultés rencontrées
- Résultats : courbes `loss/accuracy` (captures d'écran), métriques finales, matrice de confusion
- Captures d'écran de la démo Gradio en fonctionnement
- Instructions claires d'installation et d'exécution (entraînement + démo)
- Répartition du travail entre les deux membres du binôme

4. Structure Recommandée du Projet

Arborescence du repo GitHub

```
bert-classification-nomdataset/  
|-- data/                <- dataset (ou script de téléchargement)  
|-- dataset.py           <- TextClassificationDataset  
|-- model.py             <- chargement / définition du modèle BERT  
|-- train.py             <- boucles train_epoch / eval_epoch + main  
|-- demo.py              <- interface Gradio  
|-- utils.py             <- métriques, seed, visualisations  
|-- requirements.txt  
`-- README.md            <- rapport (bonus)
```

5. Hyperparamètres de Référence

Vous êtes libres de modifier ces valeurs, mais vous devez justifier vos choix dans le README.

- Modèle : bert-base-uncased (anglais) / camembert-base (français) / bert-base-multilingual-cased
- Learning rate : $2e-5$ à $5e-5$ (typique pour le fine-tuning de BERT)
- Batch size : 16 ou 32 (selon la VRAM disponible)
- Epochs : 3 à 5 (BERT converge vite en fine-tuning - attention à l'overfitting)
- Max length : 128 ou 256 tokens (à justifier selon la longueur de vos textes)
- Optimiseur : AdamW (avec `weight_decay=0.01`)
- Scheduler : linéaire avec warmup (optionnel mais recommandé)
- Loss : CrossEntropyLoss
- Seed : fixée pour la reproductibilité (random, numpy, torch)

6. Livrables Attendus

6.1 Repo GitHub (livrable unique)

- Tous les fichiers .py structurés selon la hiérarchie de la section 4
- Code commenté : chaque fonction doit avoir une docstring
- Le fichier demo.py fonctionnel (l'interface Gradio doit se lancer sans erreur)
- README.md avec au minimum : noms du binôme, dataset choisi, instructions d'exécution
- Commits des deux membres visibles dans l'historique

6.2 Envoi

Modalités de rendu

Date limite : Mardi 16 Juin 2026 - 18h00

Envoyez le **lien de votre repo GitHub** par email à : **abdouaziz@gmail.com**

Objet du mail : [Devoir 3 - BERT] Nom1 & Nom2 - NomDataset

Un seul email par binôme.

7. Conseils et Points de Vigilance

Erreurs fréquentes à éviter

- Oublier le masque d'attention lors de la tokenization → résultats dégradés
- Utiliser un learning rate trop élevé ($1e-3$) → BERT "oublie" son pré-entraînement (catastrophic forgetting)
- Oublier de basculer le modèle en mode évaluation pendant la validation
- La fonction de loss attend des labels entiers, pas des one-hot
- Ne pas fixer la seed → résultats non reproductibles
- Pousser le dataset ou les checkpoints lourds sur GitHub → utilisez un .gitignore

Bonnes pratiques

- Testez chaque module indépendamment avant de tout assembler
- Utilisez tqdm pour les barres de progression
- Commitez régulièrement avec des messages explicites - le travail des deux membres doit être visible
- Testez votre demo.py sur une machine "propre" (pip install -r requirements.txt) avant le rendu
- Consultez la documentation officielle : pytorch.org, huggingface.co/docs, gradio.app

Tout plagiat entraînera la note de 0. Le code doit être personnel et commenté. Bonne chance !